

Sistema de Generacion Automatica de Horarios Universitarios

Documentacion Tecnica Completa

Universidad Politecnica de Victoria

Estructura de Datos - Proyecto Final

Diciembre 2025

Índice

1. Introduccion	3
1.1. Objetivo del Sistema	3
2. Arquitectura del Sistema	3
2.1. Diagrama de Arquitectura General	3
3. Tecnologias y Lenguajes Utilizados	4
3.1. Stack Tecnologico Completo	4
3.2. Por que estos Lenguajes?	4
3.2.1. Python (Backend)	4
3.2.2. Cython/C (Motor de Scheduling)	4
3.2.3. JavaScript/React (Frontend)	4
4. Estructuras de Datos Utilizadas	4
4.1. Matrices de Ocupacion (C Arrays)	4
4.2. Diccionarios y Hash Maps	5
4.3. Listas Enlazadas (Implicitas)	5
5. Algoritmo de Generacion de Horarios	5
5.1. Tipo de Algoritmo	5
5.2. Por que NO Backtracking Puro?	6
5.3. Estrategia del Algoritmo	6
5.4. Heuristicas Utilizadas	6
5.5. Diagrama de Flujo del Algoritmo	7
6. Validacion de Restricciones	7
6.1. Funciones de Validacion en Cython	7
6.2. Complejidad de Validacion	8
7. Modelo de Base de Datos	8
7.1. Diagrama Entidad-Relacion	8
7.2. Tablas Principales	9

8. Backend - FastAPI	9
8.1. Arquitectura del Backend	9
8.2. Endpoints Principales	9
8.3. Flujo de Datos	10
9. Frontend - React	10
9.1. Estructura de Componentes	10
9.2. Comunicacion con Backend	10
10.Cython - Motor de Scheduling	11
10.1. Que es Cython?	11
10.2. Proceso de Compilacion	11
10.3. Optimizaciones en Cython	11
10.4. Por que C y no C++?	11
11.Complejidad Computacional	12
11.1. Analisis de Complejidad	12
12.Conclusion	12

1. Introduccion

Este documento describe la arquitectura, tecnologías, algoritmos y estructuras de datos utilizadas en el Sistema de Generacion Automatica de Horarios Universitarios. El sistema permite gestionar planes de estudio, docentes y generar horarios sin conflictos de manera automatizada.

1.1. Objetivo del Sistema

Generar horarios universitarios que cumplan con las siguientes restricciones:

- Sin empalmes de horario para maestros (un maestro no puede estar en dos lugares a la vez)
- Sin empalmes de horario para grupos (un grupo no puede tener dos clases simultaneas)
- Respetar la disponibilidad de dias de cada maestro
- Respetar el maximo de horas diarias por maestro
- Distribuir las horas de cada materia en diferentes dias de la semana
- Cumplir con las horas semanales requeridas por cada materia

2. Arquitectura del Sistema

El sistema utiliza una arquitectura de tres capas (3-tier) con separacion clara entre frontend, backend y base de datos.

2.1. Diagrama de Arquitectura General

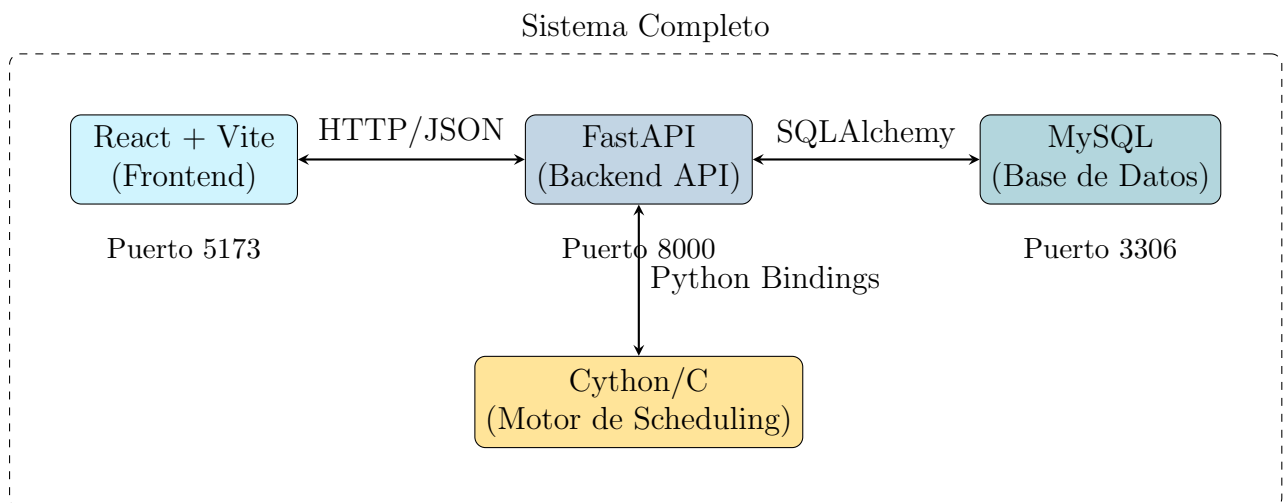


Figura 1: Arquitectura de tres capas del sistema

3. Tecnologías y Lenguajes Utilizados

3.1. Stack Tecnológico Completo

Capa	Tecnología	Lenguaje	Funcion
Frontend	React 18	JavaScript/JSX	Interfaz de usuario interactiva
Frontend	Vite	JavaScript	Bundler y servidor de desarrollo
Backend	FastAPI	Python	API REST, endpoints, validacion
Backend	SQLAlchemy	Python	ORM para base de datos
Backend	Pydantic	Python	Validacion de esquemas de datos
Motor	Cython	Python/C	Algoritmo de scheduling optimizado
Base de Datos	MySQL	SQL	Persistencia de datos

Cuadro 1: Stack tecnologico por capa

3.2. Por que estos Lenguajes?

3.2.1. Python (Backend)

- **Facilidad de desarrollo:** Sintaxis clara y rapido prototipado
- **Ecosistema:** FastAPI, SQLAlchemy, Pydantic son bibliotecas maduras
- **Integracion con C:** Python permite llamar codigo C/Cython facilmente

3.2.2. Cython/C (Motor de Scheduling)

- **Rendimiento:** El algoritmo de scheduling requiere operaciones intensivas
- **Tipos estaticos:** Cython permite declarar tipos para optimizar
- **Matrices C:** Uso de arrays C nativos para ocupacion de horarios
- **Velocidad:** Hasta 100x mas rapido que Python puro

3.2.3. JavaScript/React (Frontend)

- **Interactividad:** React permite interfaces reactivas y dinamicas
- **Componentes:** Arquitectura modular y reutilizable
- **Ecosistema:** Gran cantidad de herramientas y documentacion

4. Estructuras de Datos Utilizadas

4.1. Matrices de Ocupacion (C Arrays)

El nucleo del algoritmo utiliza matrices tridimensionales implementadas en C para rastrear la ocupacion de horarios.

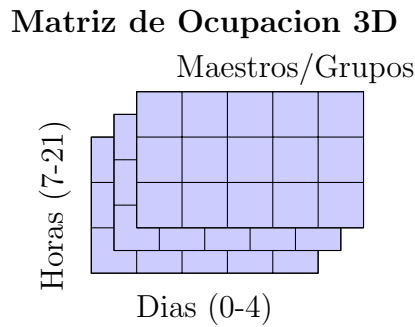


Figura 2: Representacion de la matriz de ocupacion tridimensional

Listing 1: Declaracion de matrices en Cython

```
# Matrices de ocupacion en Cython (compilado a C)
cdef int [5][14][100] ocupacion_maestros # [dia][hora][maestro_id]
cdef int [5][14][100] ocupacion_grupos # [dia][hora][grupo_id]
cdef int [100][5] horas_maestro_dia # [maestro_id][dia]
```

4.2. Diccionarios y Hash Maps

Se utilizan diccionarios de Python para:

- `maestros_por_materia`: Mapeo `materia_id` → lista de maestros
- `maestro_por_materia_grupo`: Mapeo `materia_id` → maestro asignado
- `materias_maestro_grupo`: Mapeo `maestro_id` → materia que imparte

4.3. Listas Enlazadas (Implicitas)

Las relaciones many-to-many en la base de datos actuan como listas enlazadas:

- Un maestro puede impartir multiples materias
- Una materia puede ser impartida por multiples maestros
- Un maestro tiene disponibilidad en multiples dias

5. Algoritmo de Generacion de Horarios

5.1. Tipo de Algoritmo

El sistema utiliza un **algoritmo Greedy (Voraz) con Heuristicas** en lugar de Backtracking puro. La razon principal es la eficiencia: un algoritmo de backtracking completo seria demasiado lento para horarios universitarios reales.

5.2. Por que NO Backtracking Puro?

- **Complejidad exponencial:** El espacio de búsqueda es $O(m^{n \cdot h})$ donde m = maestros, n = materias, h = horas
- **Tiempo prohibitivo:** Para 30 maestros, 58 materias y 35 horas semanales, el backtracking tardaría horas
- **Solución práctica:** Un algoritmo greedy con buenas heurísticas encuentra soluciones válidas en milisegundos

5.3. Estrategia del Algoritmo

Algorithm 1 Algoritmo Greedy de Generación de Horarios

```
1: Entrada: Lista de maestros, materias, grupos
2: Salida: Lista de asignaciones (horario completo)
3: for cada grupo do
4:   Crear mapeo maestro  $\rightarrow$  materia (un maestro solo da una materia por grupo)
5:   Ordenar materias por horas semanales (mayor primero)
6:   for cada materia del grupo do
7:     horas_restantes  $\leftarrow$  horas_semanales de la materia
8:     dias_asignados  $\leftarrow \emptyset$ 
9:     while horas_restantes  $> 0$  do
10:      mejor_dia  $\leftarrow$  día con menor carga que no tenga esta materia
11:      if mejor_dia encontrado then
12:        Asignar 1 hora de la materia en mejor_dia
13:        Marcar ocupación en matrices
14:        horas_restantes  $\leftarrow$  horas_restantes  $-1$ 
15:      end if
16:    end while
17:  end for
18: end for
19: return asignaciones
```

5.4. Heurísticas Utilizadas

1. **Ordenamiento por créditos:** Las materias con más horas se asignan primero (tienen más restricciones)
2. **Distribución balanceada:** Se busca el día con menor carga para cada asignación
3. **Evitar repetición diaria:** Una materia no se repite en el mismo día (hasta agotar días)
4. **Asignación fija maestro-materia:** Un maestro solo imparte una materia por grupo
5. **Bloques de 1 hora:** Mejor distribución que bloques grandes

5.5. Diagrama de Flujo del Algoritmo

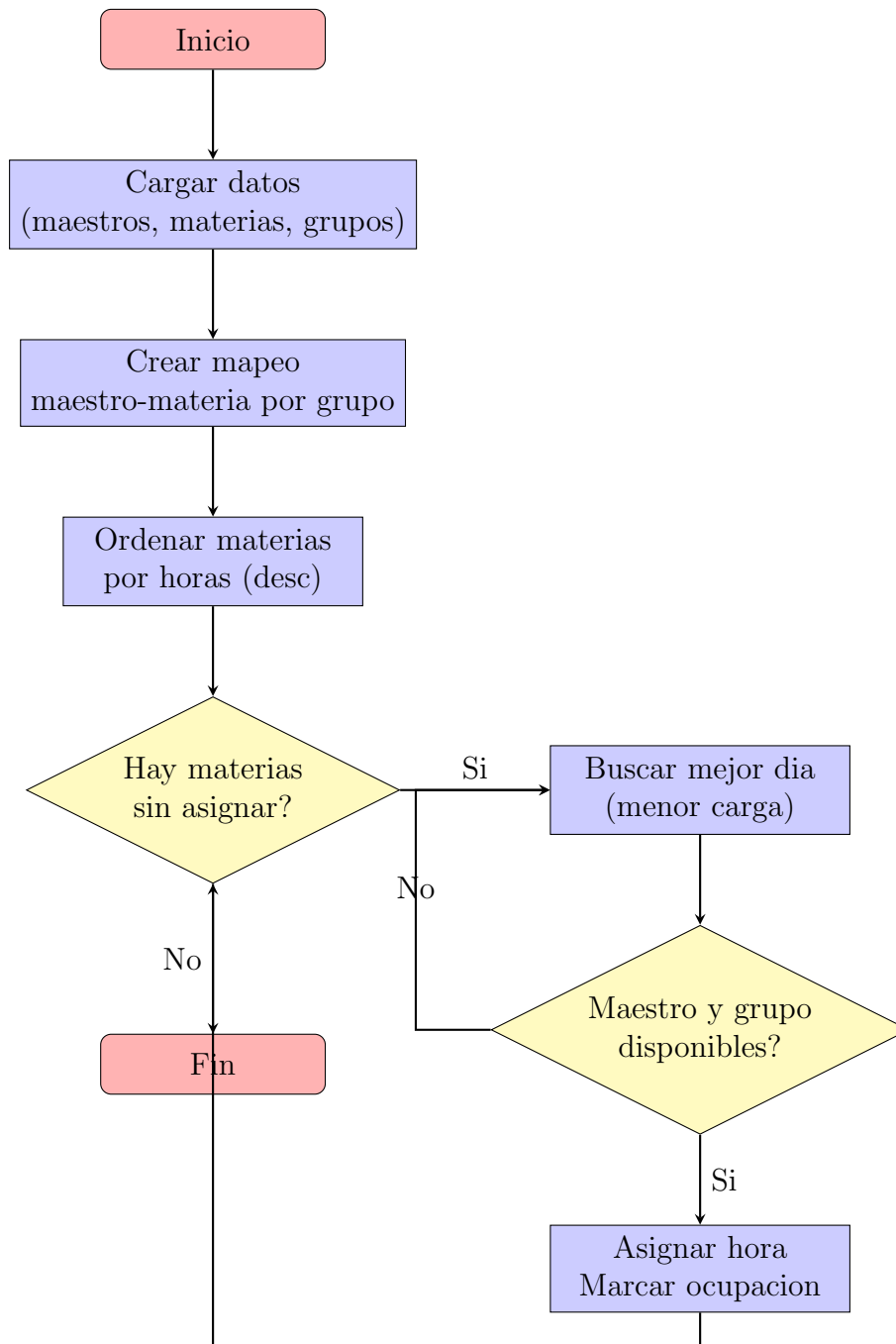


Figura 3: Flujo del algoritmo de generacion

6. Validacion de Restricciones

6.1. Funciones de Validacion en Cython

Listing 2: Validacion de disponibilidad

```
cdef bint validar_disponibilidad_maestro(self, int maestro_id,
```

```

        int dia, int
            hora_inicio,
        int hora_fin):
    """Verifica que el maestro no tenga empalme"""
    cdef int hora
    for hora in range(hora_inicio, hora_fin):
        if self.ocupacion_maestros[dia][hora - 7][maestro_id] ==
            1:
            return False # Hay empalme
    return True # Disponible

```

6.2. Complejidad de Validacion

- Verificar disponibilidad: $O(h)$ donde h = duracion del bloque
- Marcar ocupacion: $O(h)$
- Buscar mejor dia: $O(d)$ donde d = numero de dias (5)
- Acceso a matriz: $O(1)$ (acceso directo por indices)

7. Modelo de Base de Datos

7.1. Diagrama Entidad-Relacion

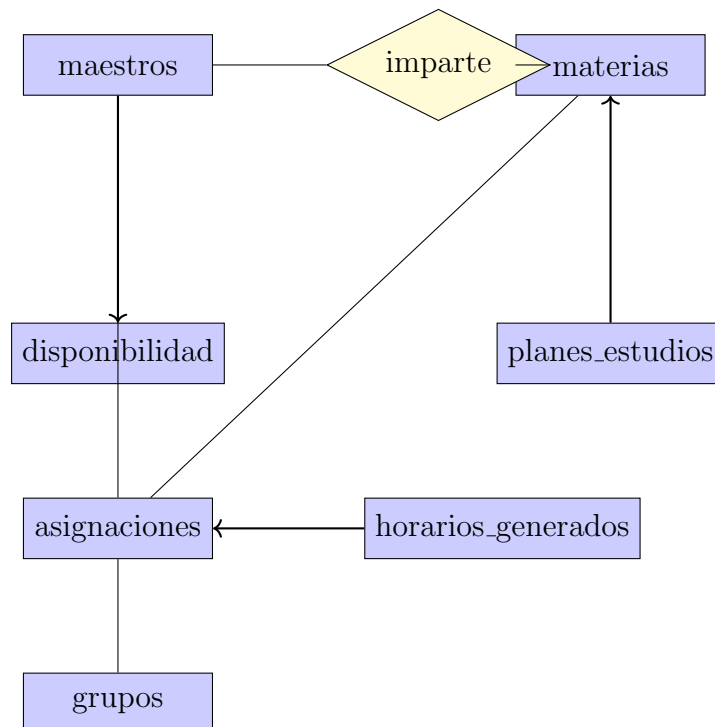


Figura 4: Modelo entidad-relacion simplificado

7.2. Tablas Principales

Tabla	Descripcion
maestros	Informacion de docentes (nombre, email, horas_max_dia)
materias	Materias del plan (nombre, horas_semanales, cuatrimestre)
planes_estudios	Planes de estudio (LITI, TSU, etc.)
grupos	Grupos por cuatrimestre (4A, 4B, etc.)
maestro_materias	Relacion N:M maestro-materia
disponibilidad_maestros	Dias disponibles por maestro
horarios_generados	Metadatos de horarios generados
asignaciones	Asignaciones individuales (maestro, materia, grupo, dia, hora)

Cuadro 2: Descripcion de tablas principales

8. Backend - FastAPI

8.1. Arquitectura del Backend

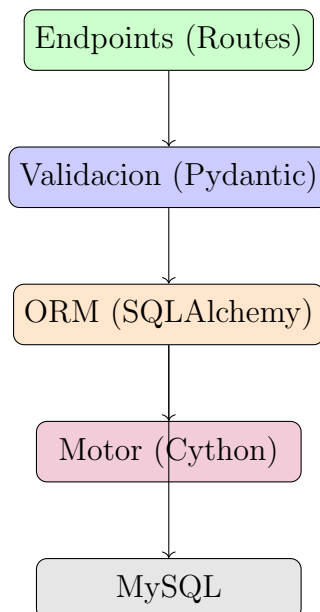


Figura 5: Capas del backend

8.2. Endpoints Principales

Listing 3: Endpoints de la API

# Maestros		
POST	/api/maestros	- Crear maestro
GET	/api/maestros	- Listar maestros
PUT	/api/maestros/{id}	- Actualizar maestro

```

DELETE /api/maestros/{id}          - Eliminar maestro
POST   /api/maestros/upload-csv    - Cargar desde CSV

# Planes de Estudio
POST   /api/planes-estudios        - Crear plan
GET    /api/planes-estudios        - Listar planes
PUT    /api/planes-estudios/{id}   - Actualizar plan
DELETE /api/planes-estudios/{id}   - Eliminar plan
POST   /api/planes-estudios/{id}/materias - Agregar materias

# Horarios
POST   /api/generar-horario-completo - Generar horarios
GET    /api/horarios                - Listar horarios
GET    /api/horarios/{id}           - Obtener horario

```

8.3. Flujo de Datos

1. El frontend envia una peticion HTTP (JSON)
2. FastAPI valida los datos con Pydantic
3. SQLAlchemy ejecuta queries a MySQL
4. Para generar horarios, se llama al modulo Cython
5. La respuesta se serializa a JSON y se envia al frontend

9. Frontend - React

9.1. Estructura de Componentes

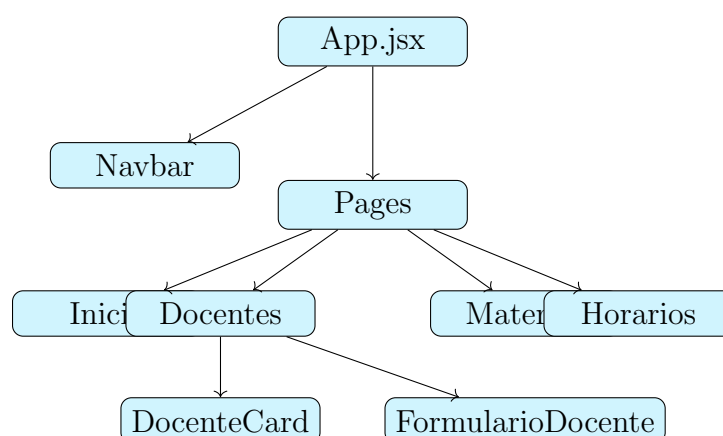


Figura 6: Arbol de componentes React

9.2. Comunicacion con Backend

Listing 4: Ejemplo de fetch al backend

```
const fetchMaestros = async () => {
    const response = await fetch('http://localhost:8000/api/
        maestros');
    const data = await response.json();
    setMaestros(data.maestros);
};
```

10. Cython - Motor de Scheduling

10.1. Que es Cython?

Cython es un lenguaje que permite escribir extensiones de C para Python. El código se escribe con sintaxis similar a Python pero se compila a C, obteniendo velocidad nativa.

10.2. Proceso de Compilacion

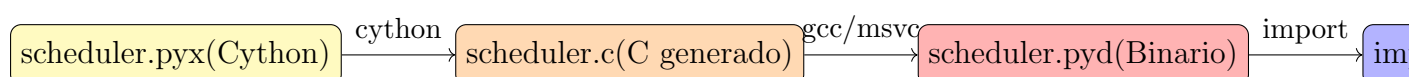


Figura 7: Pipeline de compilacion Cython

10.3. Optimizaciones en Cython

Listing 5: Tipos estaticos en Cython

```
# Variables con tipo (compiladas a C)
cdef int dia, hora, maestro_id
cdef bint disponible # booleano C

# Matrices C nativas (no listas Python)
cdef int[5][14][100] ocupacion_maestros

# Funciones C (no llamables desde Python)
cdef bint validar_disponibilidad(...):
    ...

# Funciones hibridas (llamables desde Python)
cpdef list generar_horario(...):
    ...
```

10.4. Por que C y no C++?

- **Simplicidad:** El algoritmo no requiere OOP compleja
- **Compatibilidad:** Cython genera C puro por defecto

- **Rendimiento:** C puro tiene menos overhead que C++
- **Arrays:** Se usan arrays C estaticos, no `std::vector`

11. Complejidad Computacional

11.1. Analisis de Complejidad

Operacion	Complejidad	Descripcion
Verificar disponibilidad	$O(1)$	Acceso directo a matriz
Asignar hora	$O(h)$	Marcar h slots
Generar horario grupo	$O(m \cdot d \cdot h)$	m materias, d dias, h horas
Generar horario completo	$O(g \cdot m \cdot d \cdot h)$	g grupos adicional

Cuadro 3: Complejidad de operaciones principales

Donde tipicamente: $g = 8$ grupos, $m = 7$ materias/cuatrimestre, $d = 5$ dias, $h = 8$ horas/dia.

Complejidad total: $O(8 \cdot 7 \cdot 5 \cdot 8) = O(2240) \approx O(1)$ constante para un cuatrimestre.

12. Conclusion

El sistema combina multiples tecnologias de manera eficiente:

- **React** proporciona una interfaz de usuario moderna y reactiva
- **FastAPI** ofrece una API REST rapida con validacion automatica
- **SQLAlchemy** abstrae la complejidad de la base de datos
- **Cython/C** permite un algoritmo de scheduling de alto rendimiento
- **MySQL** proporciona persistencia confiable de datos

El algoritmo greedy con heurísticas es la elección correcta para este problema, ya que:

- Encuentra soluciones validas en tiempo constante
- Las heurísticas producen horarios balanceados
- Es lo suficientemente flexible para restricciones reales
- El uso de matrices C garantiza acceso $O(1)$ para validaciones